

Universidade de Aveiro

Mestrado em Cibersegurança

AAS Pratical Project

Ano Letivo 2024/25

Lucas Rebelo Nº Mecanográfico 123934,

Tomás Bogalho Nº Mecanográfico 124224

1. Introdução

O presente projeto foi desenvolvido no âmbito da unidade curricular de Aprendizagem Aplicada à Segurança. O objetivo do projeto é classificar um ficheiro .eml (ficheiro de email) com base em dois modelos, um apenas utilizando o conteúdo do email e outro utilizando alguns cabeçalhos do email.

Para tal, o grupo decidiu criar o próprio *dataset* de emails, categorizados como *spam* ou *ham*. A aplicação final será executada através da linha de comandos.

2. Aquisição de Dados

Uma vez que o grupo decidiu criar o próprio *dataset*, foi necessária uma recolha extensiva de emails. Para poder considerar os emails de *ham*, o processo foi o seguinte, foi estabelecido o contacto com cerca de 30 pessoas diferentes e foram pedidos emails que estivessem na caixa principal do servidor de email utilizado. Tendo resultado num total de 533 ficheiros .eml utilizados com uma divisão de 248 emails de spam e 285 emails de ham.

Para o caso dos emails de *ham*, foi um processo idêntico, mas em vez da caixa principal foi utilizada a caixa de *spam*.

No fim a distribuição dos dados ficou da seguinte forma.

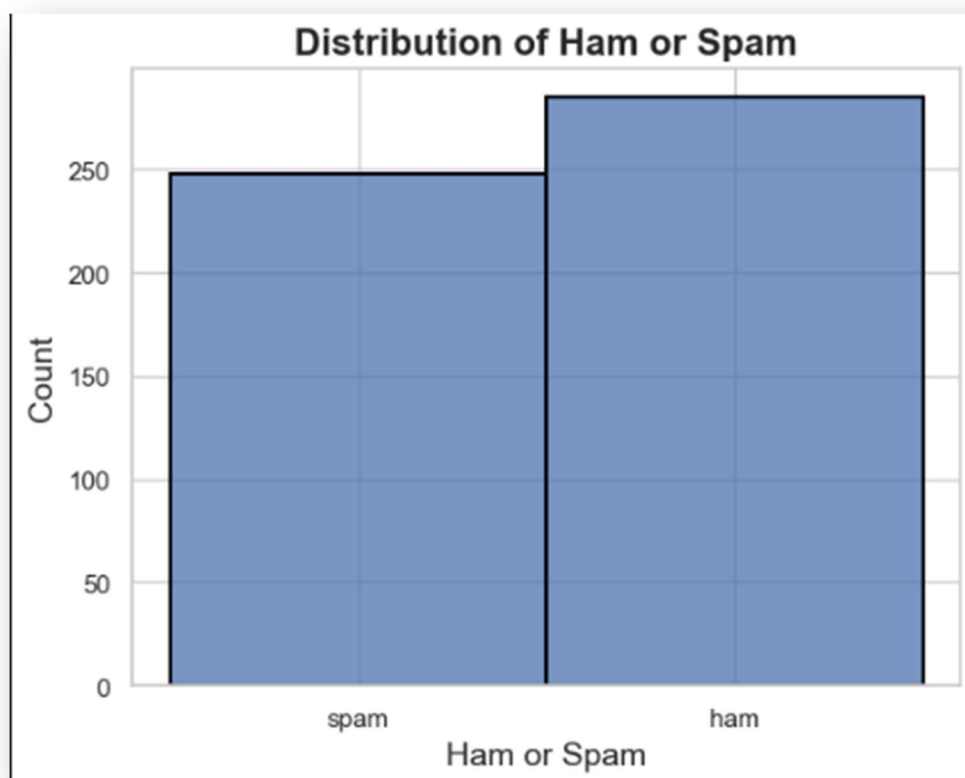


Figura 2-1 - Distribuição dos Emails Obtidos

3. Pré-Processamento

Neste capítulo será abordado o processo de pré-processamento dos conteúdos de ambos os modelos (conteúdo e de cabeçalhos) de forma a explicar o que é feito aos dados para poderem ser utilizados mais tarde pelos modelos.

3.1. Pré-Processamento do Conteúdo

Para o pré-processamento do conteúdo das mensagens de e-mail, foram desenvolvidas algumas funções auxiliares (*helper functions*), que permitem extrair e corrigir possíveis problemas de codificação e formato do texto. As operações realizadas foram as seguintes:

- **Limpeza de conteúdo HTML** - Foi criada a função `clean_html()`, que utiliza a biblioteca **BeautifulSoup** para processar o conteúdo HTML presente nas mensagens. Esta função remove as *tags* HTML, deixando apenas o texto, o que facilita o processamento posterior.
- **Correção de texto mal codificado** - Para lidar com problemas de codificação de texto, foi desenvolvida a função `fix_misencoded_text()`. Esta função tenta detetar a codificação do texto, utilizando a biblioteca **chardet**, e se necessário, ajusta a codificação do texto para uma forma legível. A função trata casos em que o texto pode ter sido mal codificado ao ser extraído de diferentes fontes ou formatos, e garante que o texto seja decodificado corretamente, usando a normalização **Unicode** para resolver potenciais problemas de caracteres especiais.
- **Extração de conteúdo de mensagens multipart** - Para lidar com mensagens *multipart* (e-mails que contêm várias partes, como texto e HTML), foi criada a função `extract_content_from_multipart()`. Esta função:
 - Verifica se a mensagem é *multipart*.
 - Se for, itera sobre as partes do e-mail e tenta extrair o conteúdo HTML ou texto, dependendo do tipo de cada parte.
 - Se houver uma parte HTML, o conteúdo é processado pela função `clean_html()` para garantir que somente o texto seja extraído.
 - Se houver uma parte de texto simples, o conteúdo é decodificado diretamente.
 - Caso a mensagem não seja *multipart*, o conteúdo é extraído diretamente, sendo também corrigido por meio da função `fix_misencoded_text()` caso necessário.

A função principal que utiliza todas as anteriores, para a obtenção do conteúdo dos e-mails foi projetada para processar ficheiros *.eml* armazenados numa pasta específica. O processo é descrito da seguinte forma:

- **Leitura e Processamento dos Ficheiros .eml**: O código começa com a definição do caminho da pasta contendo os ficheiros *.eml*, no qual foi especificado o diretório "spam_emails". Cada ficheiro *.eml* dentro dessa pasta foi lido e processado para extrair seu conteúdo.
- **Gravação num ficheiro CSV**
- **Processamento de Cada Ficheiro .eml**: Cada ficheiro é processado pela biblioteca **ByteParser** para extrair os campos necessários. De seguida é utilizada a função `extract_content_from_multipart()`, para extrair o conteúdo.

Após a obtenção dos dados, foi realizado um processo leve de *data cleaning* para garantir que o conteúdo estivesse adequado para análise. As seguintes operações de limpeza foram aplicadas ao *dataset*:

- **Remoção de Caracteres Não-Texto:** Foi desenvolvida a função `clean_text()` para limpar o conteúdo textual. Esta função realiza as seguintes tarefas:
 - Normaliza o texto utilizando a forma NFKC (compatibilidade de caracteres), o que ajuda a padronizar os caracteres com variações de codificação.
 - Substitui múltiplos espaços e quebras de linha por um único espaço, utilizando a expressão regular `\s+`.
 - Remove caracteres não alfanuméricos e símbolos que não acrescentam significado ao conteúdo, como pontuação desnecessária ou caracteres especiais (mantendo apenas letras, números, espaços, pontuação básica e alguns sinais de pontuação úteis como ponto final, vírgula, interrogação, exclamação e hífen).
- **Remoção de Duplicados:** Após o processo de limpeza do texto, foi aplicado um filtro para remover linhas duplicadas no *dataset*. A função `drop_duplicates()` foi utilizada na coluna **content**, garantindo que cada conteúdo textual fosse único. Isso é essencial, pois duplicatas no *dataset* podem ser pouco viáveis o modelo de análise ou aumentar desnecessariamente o tempo de treinamento.

3.2. Pré-Processamento dos Cabeçalhos

Para o pré-processamento dos cabeçalhos foram desenvolvidas a função *helper*:

- **extract_certifying_authorities()** - Identifica os métodos de autenticação usados para validar a mensagem de e-mail, como SPF, DKIM ou DMARC. Esses métodos são fundamentais para garantir a legitimidade do remetente e evitar falsificações, sendo informações essenciais para a análise.

Além disso, para extrair os restantes campos dos ficheiros .eml, foi utilizada a biblioteca **BytesParser** da linguagem Python. Esta biblioteca permite obter, de forma prática e eficiente, vários campos relevantes do cabeçalho e do corpo da mensagem, como o remetente, destinatário, assunto e data de envio.

Para enriquecer os dados extraídos, foram criados adicionados campos que poderão ser úteis para o modelo:

- **is_no_reply:** Coluna booleana que indica se o e-mail é do tipo "*no-reply*", identificando mensagens que não esperam resposta (geralmente notificações automáticas ou comunicados). Este campo é derivado ao verificar se o endereço do remetente contém padrões típicos como "*no-reply*" ou "*noreply*" no nome de utilizador.
- **reply_is_same_as_from:** Coluna booleana que indica se o endereço de origem (campo *From*) é o mesmo especificado no cabeçalho *Reply-To*. Diferenças entre estes campos podem indicar redireccionamentos ou e-mails potencialmente manipulados.

De seguida foi realizado um processo idêntico ao do conteúdo, apenas escrevendo os campos necessários para a avaliação através de cabeçalhos.

O conteúdo anterior foi a obtenção dos dados diretamente dos ficheiros *.eml*, de seguida será explicada o que foi feito para *data cleaning* aos dados já obtidos.

Durante o processo de limpeza de dados, foram realizadas as seguintes operações:

- **Remoção de duplicados em células** - A coluna **certifying_authorities** foi submetida a um tratamento para remover entradas duplicadas dentro de cada célula. Para tal, foi criada a função **clean_duplicates_in_cell()**, que divide o conteúdo da célula pelo delimitador “;”, remove espaços em branco desnecessários, elimina duplicados mantendo a ordem original, recombina os itens numa única string. Isto foi necessário porque por vezes um email continha valores duplicados que não faziam sentido utilizar para treinar o modelo
- **Substituição de valores ausentes:** Foram tratadas as colunas com valores em falta para evitar inconsistências:
 - Na coluna **reply_to_header**, os valores ausentes foram substituídos por *"unknown"*, indicando que o campo não estava presente no e-mail
 - Na coluna **reply_is_same_as_from**, os valores ausentes foram substituídos por **False**, assumindo que a ausência do campo implica que os endereços não são os mesmos.
 - Na coluna **certifying_authorities**, os valores ausentes foram substituídos por *"missing"*, sinalizando a falta de informação sobre os métodos de autenticação.
- **Normalização das colunas de texto:** Para garantir consistência, foi aplicada uma transformação uniforme a todas as colunas de tipo texto.
 - Os valores foram convertidos para letras minúsculas.
 - Foram removidos os espaços em branco no início e no final de cada string.

4. Treino dos Modelos

Após a preparação e limpeza dos dados, a próxima etapa do processo consiste no treino dos modelos de machine learning. O objetivo é construir modelos capazes de classificar e-mails como "ham" ou "spam" com base nos dados extraídos, tanto dos cabeçalhos quanto do conteúdo dos e-mails. Nesta fase, diferentes algoritmos de classificação são aplicados para identificar padrões que ajudem na distinção entre e-mails legítimos e indesejados.

O treino dos modelos envolve a utilização dos conjuntos de dados já preparados, que incluem as características extraídas dos cabeçalhos e do conteúdo dos e-mails. Cada conjunto de dados será usado separadamente para treinar modelos distintos, permitindo uma análise comparativa entre o desempenho de modelos baseados em cabeçalhos e em conteúdo.

4.1. Treino do Modelo com o Conteúdo

O treino do modelo com o conteúdo dos e-mails envolve o uso de técnicas de processamento de linguagem natural (NLP) para extrair características significativas dos textos e, em seguida, aplicar algoritmos de aprendizado de máquina para realizar a classificação de e-mails em *spam* ou *ham*. Neste subcapítulo, abordaremos o pré-processamento dos textos, a escolha das representações vetoriais e a aplicação de modelos de classificação.

Inicialmente, a preparação do texto inclui a lematização das palavras, remoção de *stopwords* e tokenização, garantindo que as informações relevantes sejam extraídas de maneira eficaz. Para representar o conteúdo de forma numérica, foi utilizada a técnica de **Bag of Words**, que converte os textos em vetores de frequências de palavras ou n-gramas.

Além disso, foram testados diversos modelos de classificação, incluindo **Regressão Logística**, **K-Nearest Neighbors**, **Naive Bayes**, **Random Forest Classifier**, e **Multilayer Perceptron**, avaliando a performance de cada um com base em métricas como **accuracy**, **F1-score** e **MCC** (*Matthews Correlation Coefficient*).

4.2. Treino do Modelo com os Cabeçalhos

No treino do modelo baseado nos cabeçalhos dos e-mails, foi necessária a realização de alguns pré-processamentos adicionais nos dados para que estes fossem adequados para o modelo de *machine learning*. Entre os passos realizados, destacam-se o **encoding** das variáveis categóricas e a divisão dos dados em conjuntos de treino e teste.

O **encoding** foi realizado em várias colunas do conjunto de dados, que continham variáveis categóricas, ou seja, colunas com valores não numéricos.

- As variáveis que foram codificadas são: "ham or spam", "from_header", "reply_to_header", "is_no_reply", "reply_is_same_as_from", "content_type_header", "content_type_header" e "certifying_authorities".

Para estas variáveis categóricas, foi utilizada a técnica **OneHotEncoding**. Este método transforma cada categoria em uma nova coluna binária, onde cada coluna representa a presença ou ausência de uma categoria específica. Por exemplo, para a variável "content_type_header", as categorias como "text/html" e "text/plain" seriam transformadas em colunas separadas, com valores de 0 ou 1, dependendo da presença da categoria no e-mail.

De seguida foi realizado a divisão dos dados para teste e treino, na vez que os dados estavam preparados com as variáveis codificadas, o próximo passo foi a divisão dos dados em conjuntos de treino e teste. Para isso, foi utilizado o método **train_test_split()** da biblioteca **scikit-learn**, que permite dividir os dados de forma aleatória em dois subconjuntos: um para treino e outro para teste.

De seguida foram utilizados os seguintes modelos:

- **K-Nearest Neighbors (KNN):** Este modelo classifica os dados com base nas distâncias entre os pontos de dados. Foi utilizado o valor padrão de 5 vizinhos mais próximos para a classificação.
- **Logistic Regression:** Um modelo amplamente utilizado em problemas de classificação binária. A regressão logística tenta encontrar a melhor linha de decisão entre as classes, com a adição de regularização para melhorar o desempenho.
- **Support Vector Machine (SVM):** Este modelo é eficaz em problemas de classificação de alta dimensão e tenta maximizar a margem entre as diferentes classes.
- **Random Forest:** Um modelo baseado em múltiplas árvores de decisão, que combina os resultados de várias árvores para uma predição mais robusta.

5. Avaliação dos Modelos

Após o treino dos modelos de classificação, a avaliação de seu desempenho torna-se um passo fundamental para determinar a eficácia da solução proposta. Através de métricas de avaliação é possível comparar a performance de cada modelo e escolher o mais adequado para o problema em questão.

Neste subcapítulo, abordaremos as métricas utilizadas para avaliar os modelos treinados, como

- **Accuracy:** A proporção de previsões corretas.
- **Precision:** A capacidade do modelo de identificar corretamente os e-mails de spam, considerando apenas os e-mails classificados como spam.
- **Recall:** A capacidade do modelo de identificar todos os e-mails de spam, incluindo os que podem ser classificados incorretamente.
- **F1-Score:** A média harmônica entre precisão e *recall*, que fornece uma medida equilibrada de desempenho.
- **AUC-ROC:** A área sob a curva *Receiver Operating Characteristic* (ROC), que avalia a capacidade do modelo em discriminar entre as classes.

E serão discutidos os resultados obtidos para cada um dos modelos testados. Além disso, será analisado o modelo final selecionado, que demonstrou o melhor desempenho e será utilizado na aplicação final para realizar as predições.

5.1. Avaliação do Modelo com o Conteúdo

O modelo que apresentou o melhor resultado geral foi guardado num ficheiro pickle, que neste caso foi **Random Forest Classifier** como demonstrado na seguinte figura.

```
LR Accuracy: 0.90, F1-Score: 0.90, MCC: 0.79
KNN Accuracy: 0.87, F1-Score: 0.87, MCC: 0.73
NB Accuracy: 0.92, F1-Score: 0.92, MCC: 0.84
RFC Accuracy: 0.94, F1-Score: 0.93, MCC: 0.87
MLP Accuracy: 0.92, F1-Score: 0.92, MCC: 0.84
The best model (RFC) was saved with accuracy 0.94.
```

Figura 5-1 - Resultado dos Modelos com o Conteúdo

5.2. Avaliação do Modelo com os Cabeçalhos

Como mencionado anteriormente foram utilizados vários modelos diferentes, e para avaliar foram usadas as métricas anteriores.

Após testar os quatro modelos mencionados, foi verificado que o modelo de **Regressão Logística** obteve o melhor desempenho em termos de **accuracy**, **precision** e **F1-score** em vários testes utilizando colunas diferentes no processo de *encoding*, no entanto na versão final os modelos deram todos o mesmo resultado, no entanto após confirmar com o docente esta era uma característica do *dataset*. Portanto, esse modelo foi selecionado para ser utilizado na aplicação final. Para isso foi guardado num ficheiro *.pkl* através da biblioteca **pickle**.

```
Accuracy: 0.4953271028037383
Precision: 0.7142857142857143
Recall: 0.08771929824561403
F1-Score: 0.15625
AUC-ROC: 0.523859649122807
```

	precision	recall	f1-score	support
0	0.48	0.96	0.64	50
1	0.71	0.09	0.16	57
accuracy			0.50	107
macro avg	0.60	0.52	0.40	107
weighted avg	0.60	0.50	0.38	107

Figura 5-2 - Resultado dos Modelos com os Cabeçalhos

6. Aplicação Final e Utilização

A aplicação final, representada pelo script **final_model.py**, tem como objetivo classificar e-mails no formato **EML** utilizando dois modelos de classificação distintos: um para o conteúdo dos e-mails e outro para os cabeçalhos. Para isto são utilizadas duas aplicações auxiliares a **headers_model.py** e **content_model.py**, estas são apenas utilizadas para pré-processar o ficheiro **.eml** introduzido pelo utilizador da mesma forma que é realizado no treino dos modelos. A classificação é realizada com base em modelos previamente treinados, que são carregados e aplicados aos dados extraídos do e-mail.

Para utilizar a aplicação basta abrir o repositório e correr o seguinte comando: **python final_model.py teste.eml**, em que o teste.eml é o ficheiro.

```
PS C:\Users\bogal\OneDrive - ESTGOH\Mestrado\Aveiro\Analise Aplicada a Seguranca\Projeto Mail Detection\practical-project-aas-124224-123934> python final_model.py teste.eml
Processing file: teste.eml
Processing file: teste.eml
Content Prediction: ['spam']
```

Figura 6-1 - Usage of the Application without the Error from the Headers Model

7. Conclusão

O projeto desenvolvido demonstrou a eficácia da aplicação de técnicas de Machine Learning para a classificação de e-mails em categorias de *spam* e *ham*. A criação de um *dataset* foi um desafio que demorou algum tempo uma vez que para além da obtenção dos emails foi necessário todo o processo de pré-processamento e limpeza dos dados.

Os resultados obtidos destacaram a utilidade de ambos os modelos, baseados no conteúdo e nos cabeçalhos dos e-mails, com o modelo baseado no conteúdo apresentando uma vantagem em termos de precisão. No entanto a equipa acredita que com mais afinação o modelo com os cabeçalhos poderá ser mais útil no longo prazo.

A aplicação final, acessível através de linha de comandos, foi concebida para ser intuitiva e flexível, permitindo uma análise eficiente e confiável. No entanto, uma dificuldade surgiu ao tentar alimentar o modelo de cabeçalhos, relacionado com os campos gerados pelo *encoder*. Este problema impediu a utilização plena do modelo de cabeçalhos na aplicação final, limitando a sua capacidade de realizar previsões de forma eficaz. Apesar desse desafio, os resultados do modelo de conteúdo foram bem-sucedidos.

Apesar do sucesso alcançado, existem oportunidades para melhorias futuras, como a ampliação do *dataset*, a integração de modelos mais avançados e o desenvolvimento de um sistema de feedback para corrigir eventuais erros de classificação. Também seria importante revisar a parte do código responsável pela *encoding* dos cabeçalhos para garantir a plena funcionalidade do modelo associado.

8. Referencias

1. Medhat, H. (2021). *Spam Emails Classify ML*. GitHub. Disponível em: <https://github.com/HanyMedhat10/Spam-Emails-Classify-ML/blob/main/main.ipynb>
2. GeeksforGeeks. (2021). *Detecting Spam Emails using TensorFlow in Python*. GeeksforGeeks. Disponível em: <https://www.geeksforgeeks.org/detecting-spam-emails-using-tensorflow-in-python/>
3. GeeksforGeeks. (2021). *Implementing Web Scraping in Python using BeautifulSoup*. GeeksforGeeks. Disponível em: <https://www.geeksforgeeks.org/implementing-web-scraping-python-beautiful-soup/>
4. Medium. (2020). *Spam Email Detector*. Analytics Vidhya. Disponível em: <https://medium.com/analytics-vidhya/spam-email-detector-73518ba46340>
5. GeeksforGeeks. (2021). *ML - One Hot Encoding*. GeeksforGeeks. Disponível em: <https://www.geeksforgeeks.org/ml-one-hot-encoding/>
6. GeeksforGeeks. (2021). *Saving a Machine Learning Model*. GeeksforGeeks. Disponível em: <https://www.geeksforgeeks.org/saving-a-machine-learning-model/>
7. Conteúdo da UC disponibilizado pelo docente